

CUDA Matrix Multiplication: Naive vs Tiled Shared Memory vs cuBLAS

Course: Parallel Computing and GPU Programming

Author: Marcos Gómez Morena

Repository: https://github.com/ChemaCrema/Parallel_Programming_finalproject

Abstract

The goal of this project is to make an implementation of benchmarks and make an analysis of three different and better optimised GPU implementations of the general matrix multiplication also known as GEMM, which is a really intense computational operation in the topics of machine learning and scientific computing.

For the three different implementations I will be doing: a naive CUDA kernel, a tiled shared memory kernel, and NVIDIA's cuBLAS library as a state-of-the-art baseline. In order to execute this implementations I done it on an NVIDIA A100 GPU using the Mahti supercomputer, using matrix sizes whose range is from 256×256 to 4096×4096 . At the end of the experiments the tiled kernel achieves around a 2x speedup over the naive implementation by exploiting shared memory to reduce global memory traffic. Eventhough we achive a decent improvement, this results makes clear the difference between a hand written tiled kerner and the cuBLAS implementation that is optimizrd by NVIDIA.

1. Introduction

A matrix multiplication ($C = A \times B$, where $A, B, C \in \mathbb{R}^{N \times N}$) needs $2N^3$ floating point operations. So in order to do large N , the performance is bottlenecked by memory bandwidth rather than compute, with each output element requiring to read an entire row of A and column of B from global memory. We have to also take into consideration that modern GPUs have high peak FLOPS but a more limited memory bandwidth, making memory access patterns the key optimisation target.

Project goals:

- Implement a correct naive CUDA kernel as the baseline
- Implement a tiled shared memory kernel to reduce global memory traffic
- Benchmark both against cuBLAS (SOTA)
- Analyse the performance gap and explain its causes

2. Background

2.1 GPU memory hierarchy

The NVIDIA GPUs have a lot of different levels of memory that differs drastically in latencies and bandwidths:

Memory Type	Latency	Bandwidth	Size
Global memory (DRAM)	~400–800 cycles	~2 TB/s (A100)	40 GB
Shared memory (SRAM)	~20–30 cycles	~19 TB/s	164 KB/SM
Registers	~1 cycle	—	256 KB/SM

The main conclusion that we can obtain is that shared memory is roughly **20× faster** than global memory and can be explicitly managed by the programmer.

2.2 Arithmetic intensity

The arithmetic intensity also known as AI, measures the ratio of floating point operations to memory bytes transferred:

$$AI = \text{FLOPs} / \text{Bytes}$$

For matrix multiplication:

- **Naive kernel:** each output element reads $2N$ floats $\rightarrow AI = 2N^3 / (2N^2 \times 4) = N/4$ FLOP/byte
- **Tiled kernel:** tiles of size T are loaded once and reused T times $\rightarrow AI \approx T/4 \times N/4$, effectively multiplying by T

The tiled implementation increases arithmetic intensity by reusing each tile element multiple times from shared memory, reducing global memory traffic by approximately a factor of `TILE_SIZE`.

This increases the arithmetic intensity, moves the computation from memory-bound to compute-bound, which is need it to have a high utilization for the GPU.

3. Implementation

3.1 Naive kernel

Each CUDA thread computes one output element `C[row][col]` by reading a full row of `A` and a full column of `B` from global memory.

```
__global__ void matmul_naive(const float* A, const float* B, float* C, int
N) {
    int row = blockIdx.y * blockDim.y + threadIdx.y;
    int col = blockIdx.x * blockDim.x + threadIdx.x;
    if (row < N && col < N) {
        float sum = 0.0f;
```

```

        for (int k = 0; k < N; k++)
            sum += A[row * N + k] * B[k * N + col];
        C[row * N + col] = sum;
    }
}

```

The problem is that for an $N \times N$ output matrix, this kernel have a $O(N^3)$ global memory reads. With many of these reads being redundant, and having neighbouring threads in the same block read overlapping rows and columns.

3.2 Tiled shared memory kernel

The tiled kernel implementation divides the matrices into square tiles of size $TILE_SIZE \times TILE_SIZE$ (32×32 in our implementation), making each thread block cooperatively loads one tile of A and one tile of B into shared memory, performs the partial dot product, then advances to the next tile.

```

__global__ void matmul_tiled(const float* A, const float* B, float* C, int
N) {
    __shared__ float sA[TILE_SIZE][TILE_SIZE];
    __shared__ float sB[TILE_SIZE][TILE_SIZE];

    int row = blockIdx.y * TILE_SIZE + threadIdx.y;
    int col = blockIdx.x * TILE_SIZE + threadIdx.x;
    float sum = 0.0f;

    for (int t = 0; t < (N + TILE_SIZE - 1) / TILE_SIZE; t++) {
        // Cooperative load into shared memory
        sA[threadIdx.y][threadIdx.x] = A[row * N + t * TILE_SIZE +
threadIdx.x];
        sB[threadIdx.y][threadIdx.x] = B[(t * TILE_SIZE + threadIdx.y) * N
+ col];
        __syncthreads(); // Wait for all threads to finish loading

        // Accumulate partial result from shared memory
        for (int k = 0; k < TILE_SIZE; k++)
            sum += sA[threadIdx.y][k] * sB[k][threadIdx.x];
        __syncthreads(); // Wait before loading next tile
    }
    if (row < N && col < N) C[row * N + col] = sum;
}

```

Main point of the optimisation:

- Each element of A and B is loaded from global memory exactly once per tile and reused $TILE_SIZE$ times
- The `__syncthreads()` barrier make sure that all threads in the block have loaded their tile before the computation begins
- Global memory traffic is reduced by a factor of $TILE_SIZE$ ($32 \times$)

3.3 cuBLAS Baseline

As the last implementation we have cuBLAS that is NVIDIA's optimised BLAS library, that uses hardware accelerated tensor cores on A100 (available for FP16/BF16/TF32)

and uses advanced optimisations like double-buffering, software pipelining, and warp-level primitives, being a lot more optimized than a hand-written kernel.

```
cublasSgemm(handle, CUBLAS_OP_N, CUBLAS_OP_N, N, N, N,
            &alpha, d_B, N, d_A, N, &beta, d_C, N);
```

4. Experimental setup

4.1 Hardware and software configuration

Component	Details
GPU	NVIDIA A100-SXM4-40GB
Compute Capability	8.0 (Ampere)
GPU Memory	40 GB HBM2e
CPU	AMD EPYC 7H12
CUDA Version	11.5
Compilation flags	-O3 -arch=sm_80
Matrix sizes tested	256, 512, 1024, 2048, 4096
Data type	float32 (FP32)
Timing method	CUDA Events (GPU-side, excludes H2D/D2H transfers)
Tile size (tiled kernel)	32×32
Block size (naive kernel)	16×16

With each configuration being run after a warm up in order to eliminate the library initialization overhead, mainly for the cuBLAS implementations.

4.2 Benchmark methodology

In order to obtain the performance measurements I used CUDA Events, which provide GPU side timing with minimal overhead. In this measurements I only measure kernel execution time, not including host-to-device and device-to-host memory transfers from the benchmark in order to just focus on computational performance.

In order to avoid initialization overheads to affect the measurements, I used a warm-up execution before timing each implementation, being especially important for cuBLAS, whose first invocation may include library and CUDA runtime initialization costs.

For each matrix size, the benchmark allocated device memory, initialized input matrices with random values, executed the corresponding kernel, and recorded the execution time using CUDA events. Throughput was then computed in GFLOPS using the theoretical operation count of matrix multiplication ($2N^3$ floating-point operations).

In order to make sure about the correctness I compared the outputs of the tiled kernel and cuBLAS against the naive implementation using the maximum absolute element-wise difference.

5. Results

5.1 Execution time

N	Naive (ms)	Tiled (ms)	cuBLAS (ms)
256	0.03	0.03	0.04
512	0.13	0.09	0.05
1024	0.90	0.55	0.18
2048	7.02	4.11	1.26
4096	66.84	33.35	7.24

5.2 Throughput (GFLOPS)

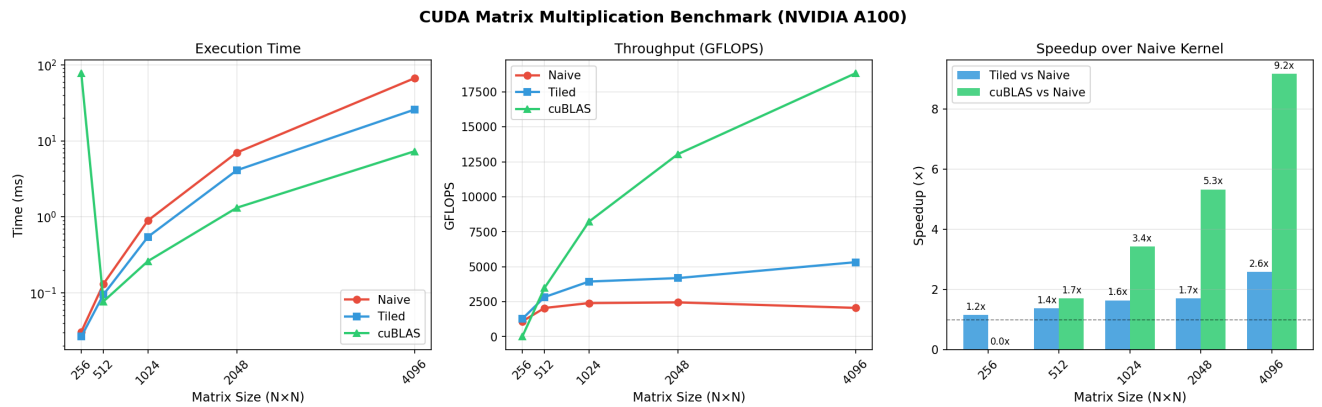
N	Naive	Tiled	cuBLAS
256	1092	1253	936
512	2032	2828	5699
1024	2397	3935	11916
2048	2449	4182	13673
4096	2056	4122	18992

The theoretical FP32 peak performance of an NVIDIA A100 is approximately 19.5 TFLOPS, in the test we obtain for the largest matrix size tested in cuBLAS around 18.99 TFLOPS, this being aprox the 97% of peak FP32 throughput.

5.3 Speedup over Naive

N	Tiled / Naive	cuBLAS / Naive
256	1.15x	0.75x
512	1.39x	2.60x
1024	1.64x	5.00x
2048	1.71x	5.57x
4096	2.00x	9.23x

5.4 Benchmark visulization



5.5 Correctness

I compared the tiled kernel and the cuBLAS outputs against the naive implementation, obtaining a maximum absolute element wise difference that is under the 10^{-4} for all matrix sizes, this confirms as the numerical correctness with a precision of FP32.

6. Analysis and Discussion

6.1 Why Does Tiling Help?

After seeing the results we can confirm that the tiled implementations beats in a consistent way the naive kernel one due to the fact that matrix elements that are loaded into the shared memory can be reutilized by multiple threads within the block, reducing the number of accesses to global memory and increasing the arithmetic intensity.

With the benefit increasing as the matrix size does too:

- $N=256 \rightarrow 1.15\times$ speedup
- $N=1024 \rightarrow 1.64\times$ speedup
- $N=4096 \rightarrow 2.00\times$ speedup

With this we can say that larger matrices become increasingly memory-bound, making data reuse more important.

6.2 Why Is cuBLAS So Much Faster?

Eventhough the tiled implementation doubles performance against the naive kernel one, cuBLAS still remains beint the fastest by a huge margin, achieving a $9.23\times$ speedup over the naive implementation in it's best performance and an approximately $4.6\times$ speedup over the tiled kernel for the $N=4096$.

This performance advantage comes from several advanced optimizations:

- Tensor Core utilization
- Register tiling
- Double buffering
- Software pipelining

- Warp-level matrix instructions (WMMA)
- Architecture-specific scheduling and memory optimizations

These techniques makes possible that cuBLAS can operate close to the theoretical limits of the A100 GPU.

6.3 Scalability

All implementations that I tested confirm the $O(N^3)$ growth in the execution time, and even though the tiled kernel scales clearly better than the naive implementation, maintaining a higher throughput as the matrix size increases, the speedup is still pretty modest (around $2\times$) if we compare it to a highly optimized production libraries, making us realize that shared memory tiling is not enough by itself in order to fully use modern GPU hardware.

7. Conclusion

With this project I was able to prove how important is the memory hierarchy optimization on GPU matrix multiplications.

The naive kernel implementation was able to achieve approx 2.0 TFLOPS for large matrices but is limited by global memory traffic when this is too much. If we use shared memory tiling instead, the performance augments to over 4.1 TFLOPS, meaning we achieve a speedup of $2.0\times$.

Nevertheless, cuBLAS was able to reach approx 19 TFLOPS on the same hardware, making really clear that additional optimizations like tensor cores, register tiling, and software pipelining are need it in order to be able to approach peak GPU performance.

After all this experiments we can conclude that in order to have an efficient GPU programming we not only need parallelism, but we also need to maximize data reuse and minimize global memory accesses due to its expense.

References

1. NVIDIA Corporation. (2023). *CUDA C++ Programming Guide*. <https://docs.nvidia.com/cuda/cuda-c-programming-guide/>
2. NVIDIA Corporation. (2023). *cuBLAS Library User Guide*. <https://docs.nvidia.com/cuda/cublas/>
3. Volkov, V. (2010). *Better performance at lower occupancy*. GPU Technology Conference.
4. Kerr, A. et al. (2017). *CUTLASS: Fast Linear Algebra in CUDA C++*. <https://github.com/NVIDIA/cutlass>
5. Harris, M. (2007). *Optimizing Parallel Reduction in CUDA*. NVIDIA Developer Technical Report.
6. CSC – IT Center for Science. (2023). *Mahti Supercomputer Documentation*. <https://docs.csc.fi/computing/systems-mahti/>